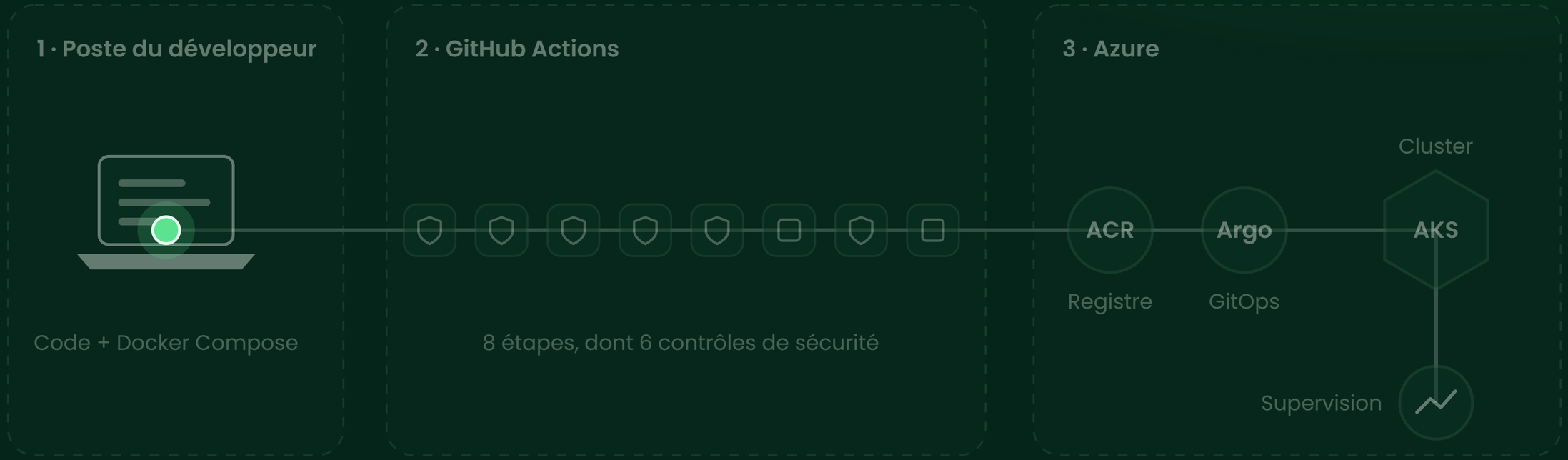
Soutenance de projet de fin d'études · ISIMa

Une chaîne DevSecOps amène le code de ZAWEDNI jusqu'au cloud

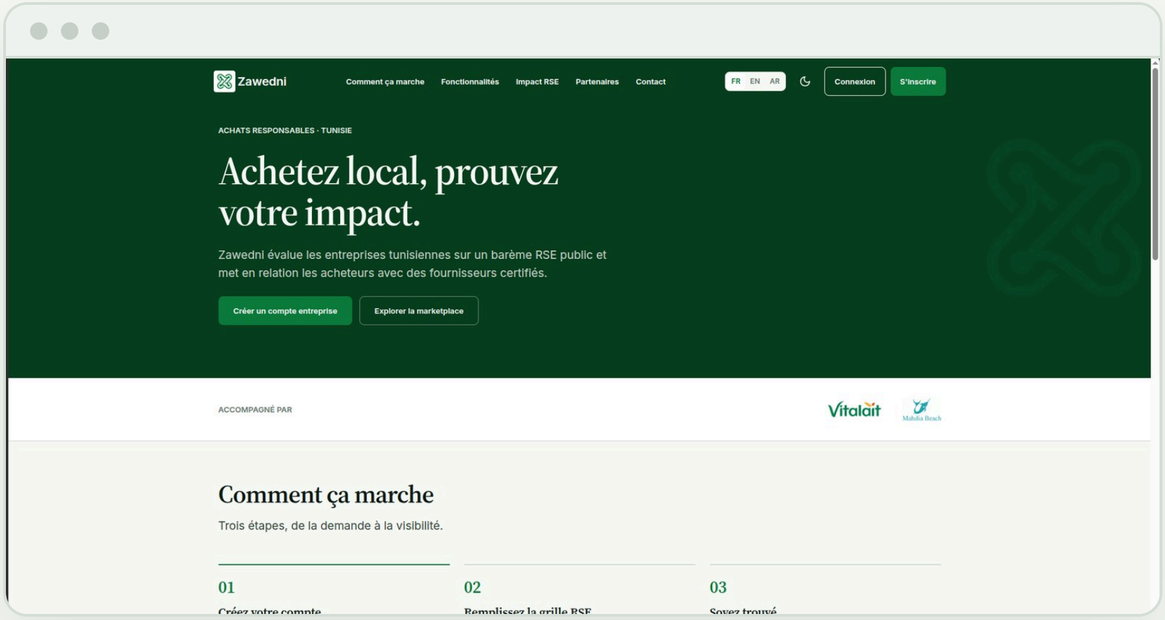
Ali Zribi · Mastère professionnel en informatique · 2025–2026



Encadrantes : Mme Amel Zidi · Mme Sana Ghannay · Mme Hadil Ben Masseoud (ZAWEDNI)

Contexte · La startup

ZAWEDNI relie des acheteurs et des fournisseurs locaux vérifiés par une grille RSE



Capture du 06/10 · site client de staging

Acheteurs

Hôtels, maisons d'hôtes

Fournisseurs

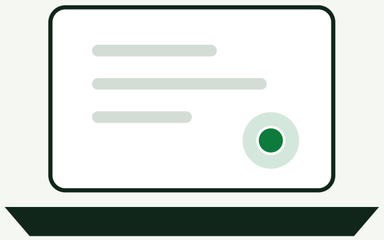
Producteurs locaux, artisans

Règle

Visible seulement après validation de la grille RSE

Contexte · Situation de départ

Au départ, le code tournait sur un seul ordinateur, avec quatre manques



0

analyses de sécurité automatiques

1

Mise en ligne

Tout se faisait à la main.

2

Sécurité

Aucun contrôle de sécurité.

3

Cloud et test

Pas d'infrastructure, pas d'environnement de test.

4

Supervision

Une panne passe inaperçue.

Contexte · Problématique

La question : livrer cinq composants en sécurité, avec un budget de startup

Livrer automatiquement cinq composants ?

Les sécuriser et les surveiller ?

Avec le budget d'une startup.

15 besoins fonctionnels

7 besoins non fonctionnels

BF05
Un contrôle bloque un problème critique

BNF07
Tout tient dans un crédit de 100 $

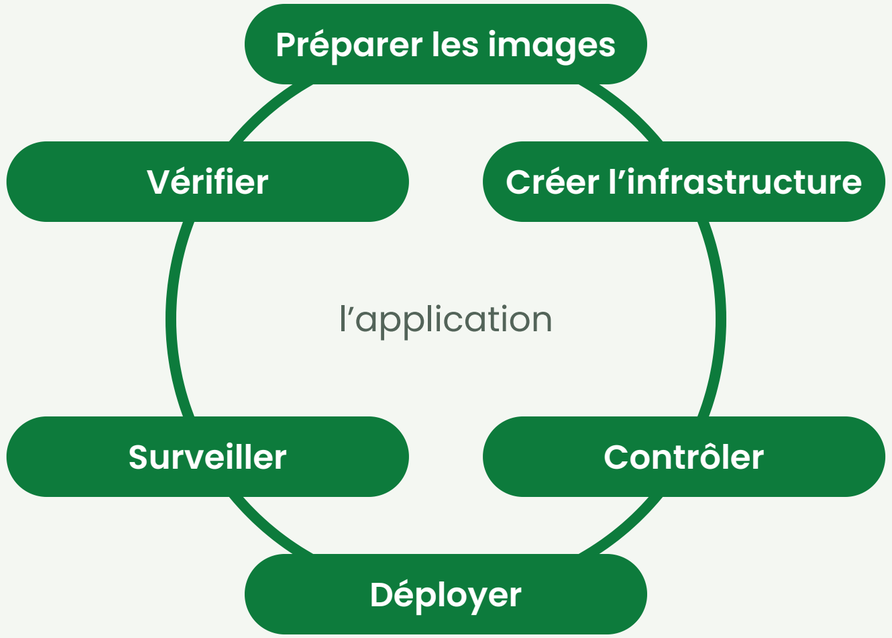
Contexte · Ma contribution

Mon travail dans ce PFE est la chaîne autour de l'application, qui existait déjà

Avant le PFE : l’application

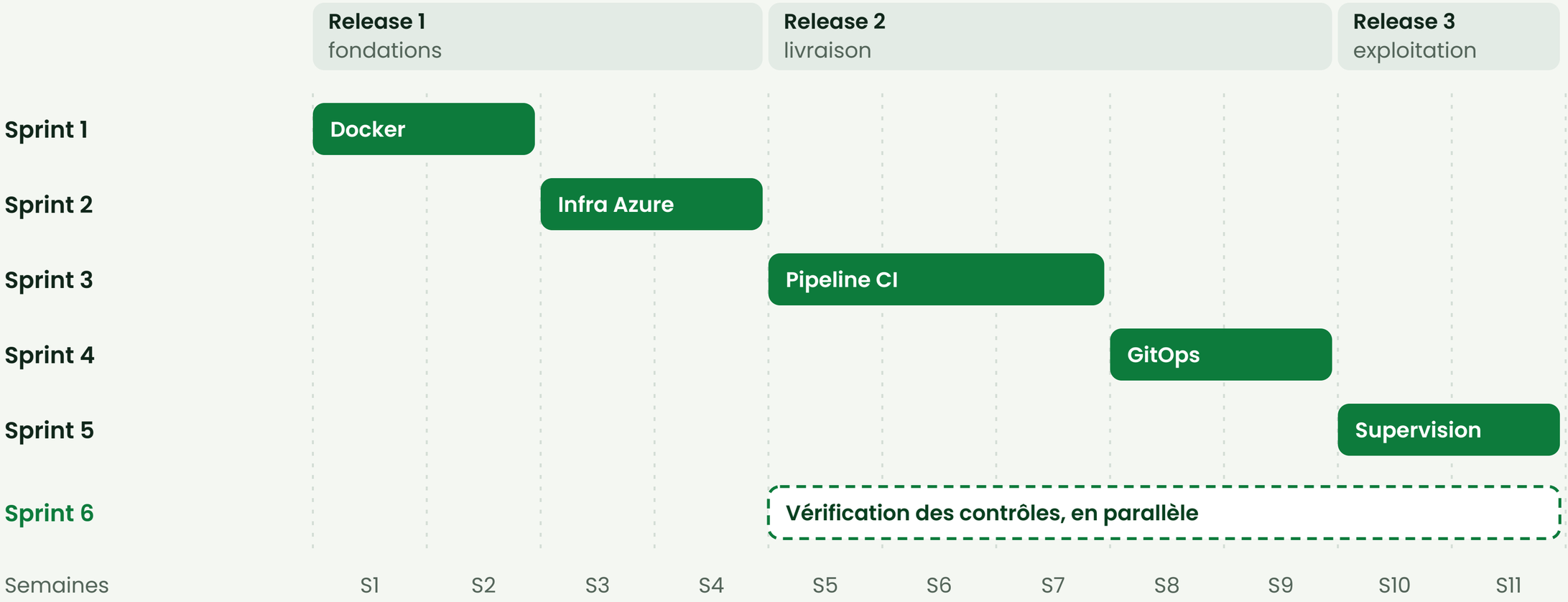
- **API** — Spring Boot
- **Site client** — React
- **Site admin** — React
- **Mobile** — Flutter
- **Service IA**
- **PostgreSQL**

Mon travail dans le PFE : la chaîne



Contexte · Méthode

Le travail a suivi Scrum : six sprints en trois releases



Architecture · Le choix

GitHub Actions automatise les contrôles ; AKS héberge les services

Choix	Besoin	Raison du choix
GitHub Actions	Tester et construire automatiquement	Dépôt déjà sur GitHub ; pas de serveur CI à entretenir
AKS sur Azure	Exécuter et organiser les services	Kubernetes géré ; crédit étudiant disponible

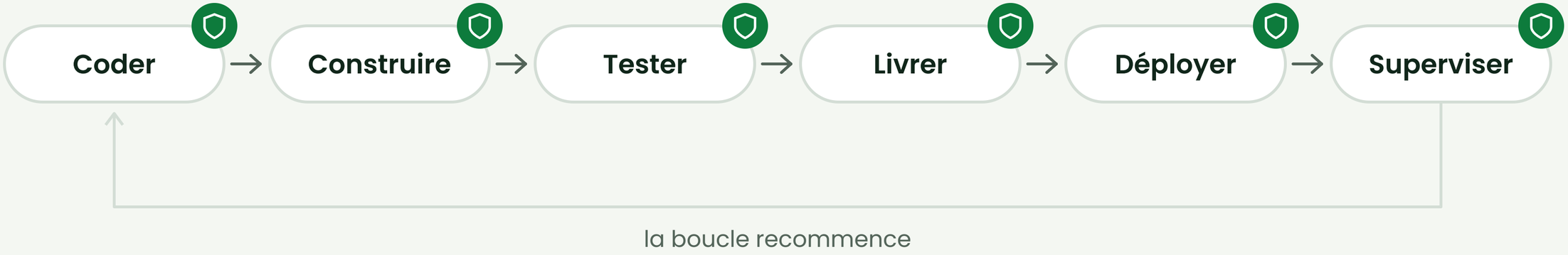
Contrainte : crédit Azure for Students de 100 $

Autres solutions possibles : Jenkins, GitLab CI, Azure Pipelines

Architecture · DevSecOps

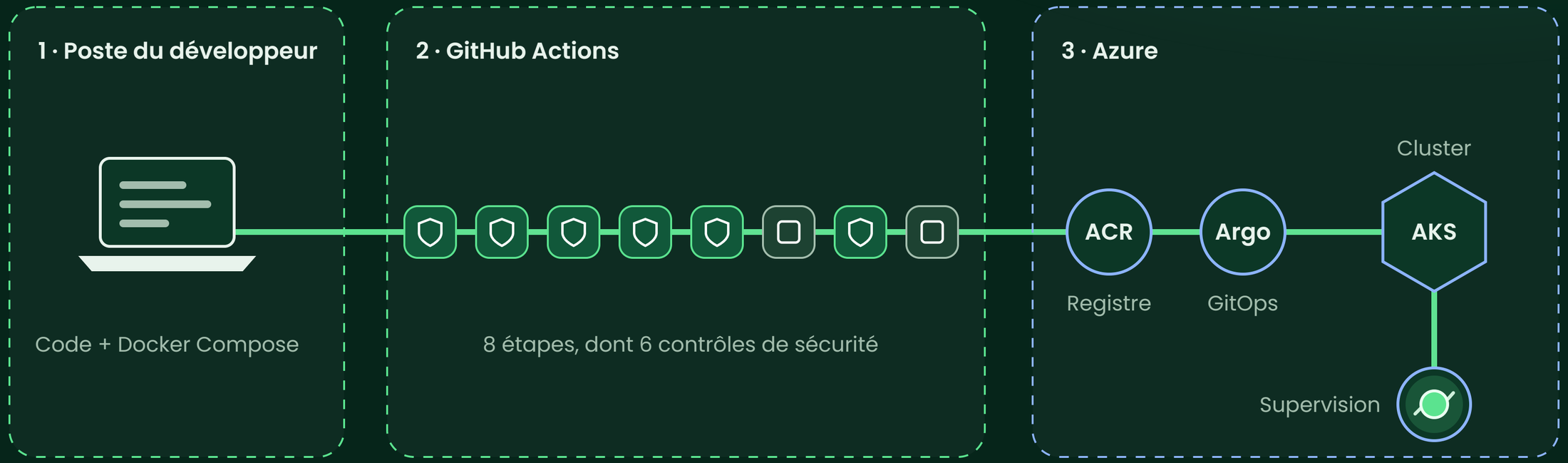
DevSecOps place les analyses de sécurité dans le pipeline, à chaque changement

DevSecOps : un contrôle dans chaque étape, à chaque changement



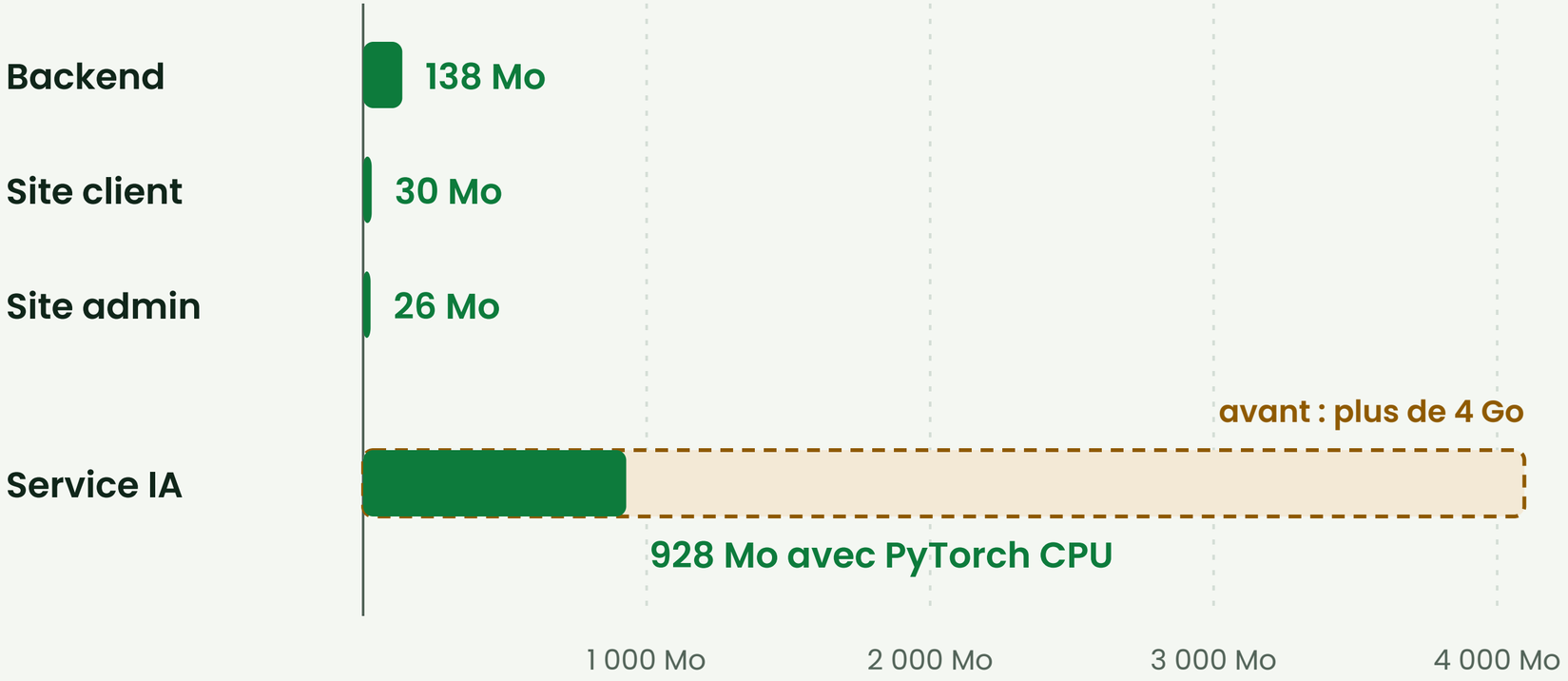
Architecture · Vue d'ensemble

La chaîne a trois zones : le poste, GitHub et Azure



Architecture · Docker

Docker met chaque service serveur dans une image qui tourne partout



Mobile

→ APK

pas d'image

Architecture · Terraform

Terraform décrit l'infrastructure Azure dans du code

main.tf

```
resource "azurerm_kubernetes_cluster" "zawedni" {
  name                = var.cluster_name
  resource_group_name = azurerm_resource_group.zawedni.name
  dns_prefix          = "zawedni"
  default_node_pool {
    name       = "default"
    node_count = 2
    vm_size    = "Standard_B2s"
  }
  identity { type = "SystemAssigned" }
}
```

main.tf
dans le dépôt Git

Décrits dans ce code :

AKS	cluster Kubernetes
ACR	registre d'images
Stockage	compte Azure

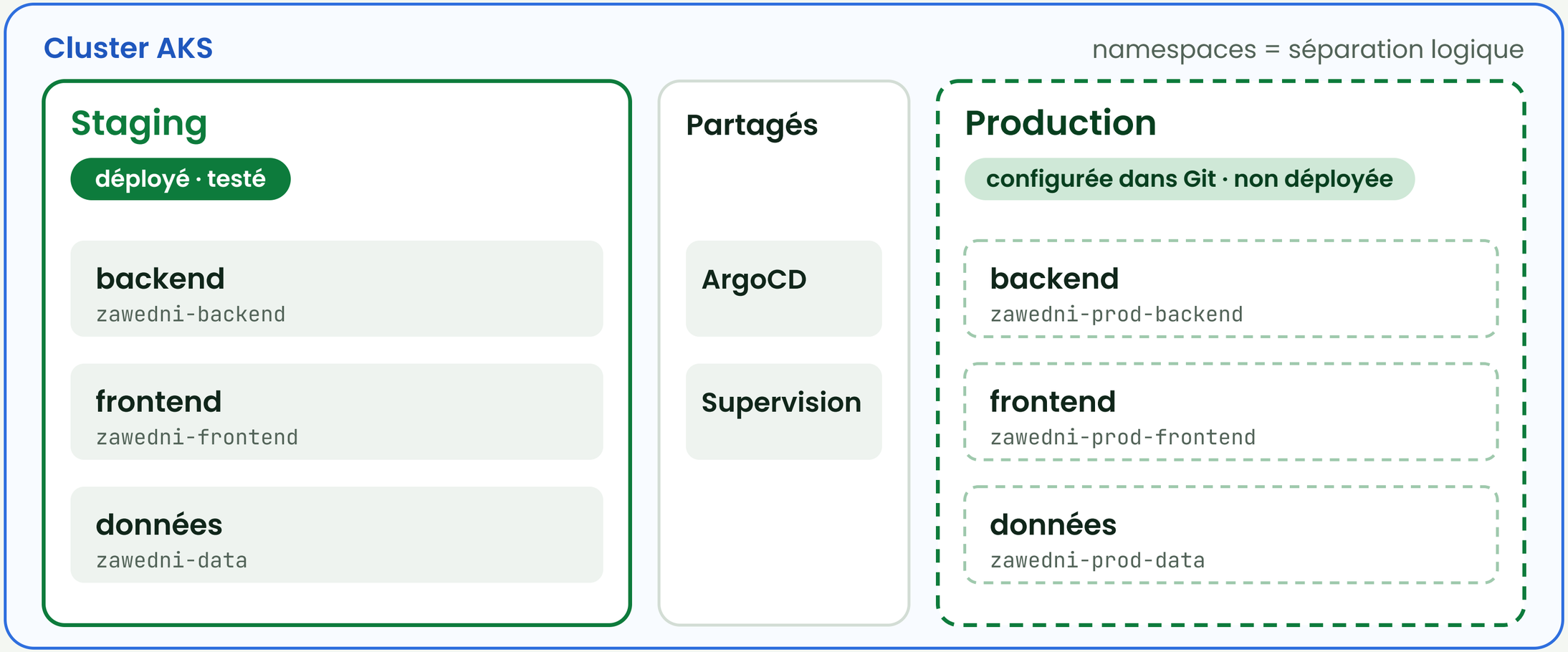
`terraform plan` on relit

`terraform apply` **on applique**

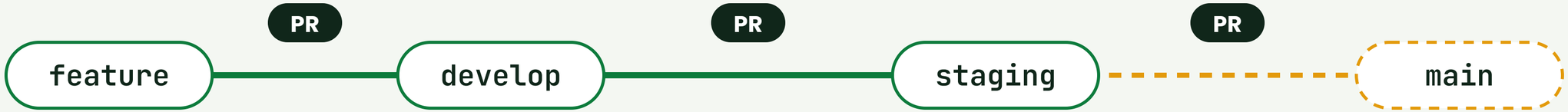
État du rapport (ligne location retirée de l'extrait) · aujourd'hui : 1 nœud Standard_B2as_v2 (variables.tf)

Architecture · Environnements

Le cluster prévoit deux environnements : staging est déployé, la production est seulement configurée



Le chemin prévu d'un changement passe par develop et staging avant main



On code ici

une branche
par changement

PR vers develop

Semgrep
aucune image

PR vers staging

contrôles des
fichiers modifiés

↓

fusion

↓

déploiement staging

↓

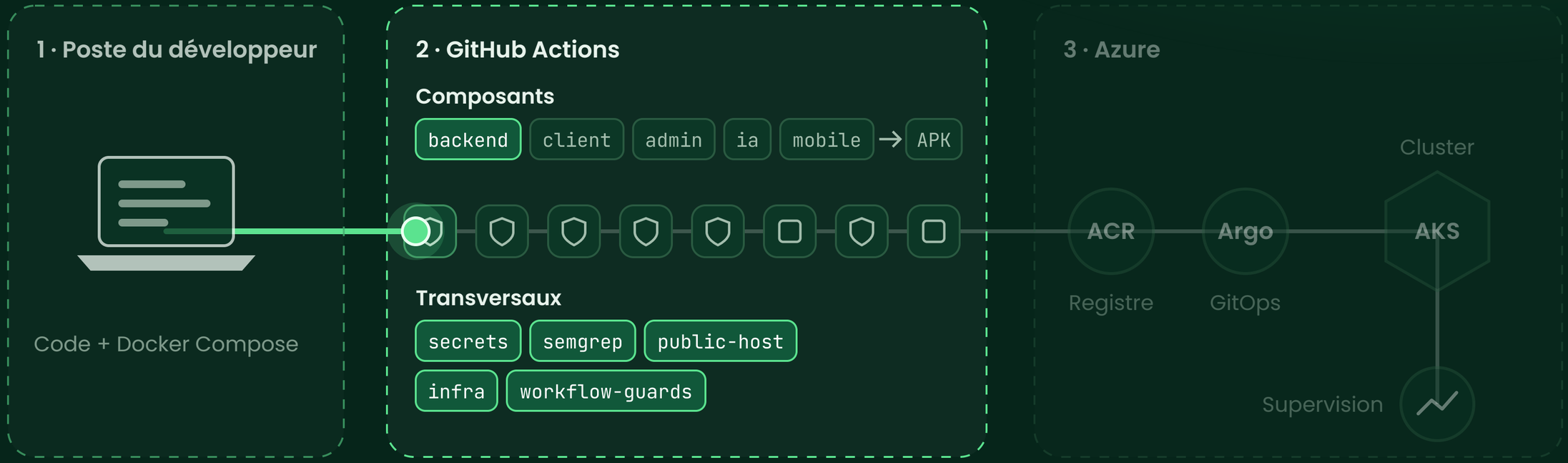
test

PR vers main

prévu ·
pas encore réalisé

Parcours d'un changement · Les pipelines

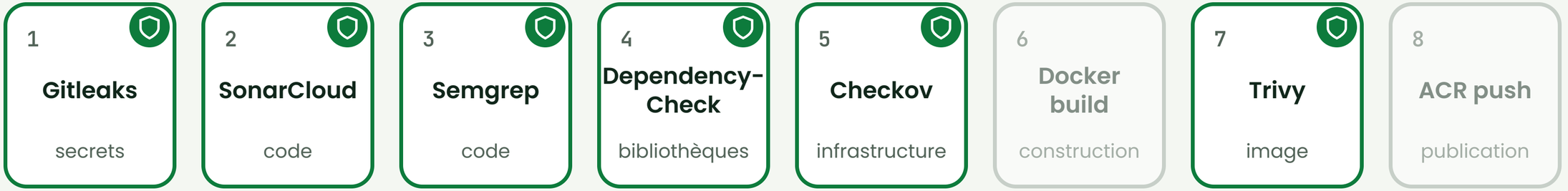
Cinq pipelines suivent un composant chacun, cinq autres vérifient tout le dépôt



Secrets, Semgrep, adresse publique : à chaque push sur staging ou main.
Infrastructure et workflows : quand leurs fichiers changent.

Parcours d'un changement · Vue de synthèse

Après les tests, les pipelines enchaînent six analyses de sécurité, la construction et la publication



Types d'analyses de sécurité dans le dépôt

6

Analyse de sécurité

Construction et publication

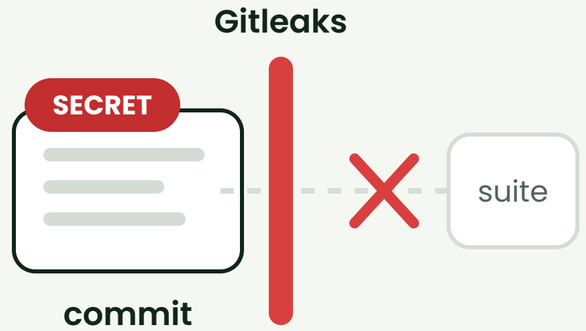
Vue globale : analyses réparties entre plusieurs workflows

Parcours d'un changement · Analyse bloquante 1 sur 3

Gitleaks fait échouer le job s'il trouve un secret dans le code

ci-backend.yaml

```yaml
- name: Gitleaks Secret Scan
  run: |
    ./gitleaks git . \
      --log-opts="${LOG_OPTS}" \
      --config .gitleaks.toml \
      --redact \
      --exit-code 1 \
      --report-format json \
      --report-path gitleaks-report.json
```



exit-code 1 : le job échoue

Rien n'est publié.

Extrait de .github/workflows/ci-backend.yaml (branche staging), lignes simplifiées

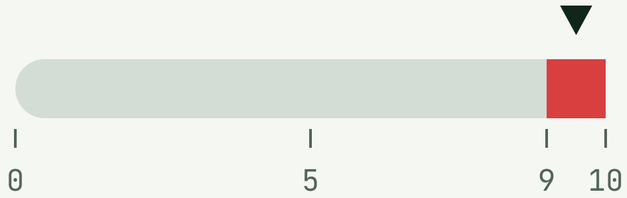
Dependency-Check fait échouer le build sur une bibliothèque avec une faille critique

pom.xml

```xml
<plugin>
  <groupId>org.owasp</groupId>
  <artifactId>dependency-check-maven</artifactId>
  <configuration>
    <failBuildOnCVSS>9</failBuildOnCVSS>
    <formats>
      <format>HTML</format>
      <format>JSON</format>
    </formats>
  </configuration>
</plugin>
```

Extrait de ZawedniServer/pom.xml (branche staging), lignes simplifiées

Score CVSS : la gravité d'une faille



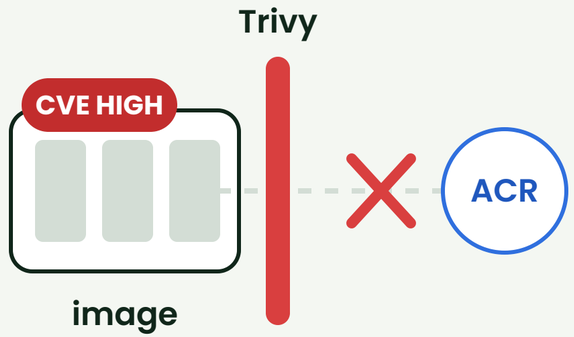
9 ou plus : faille critique

Le build échoue, rien n'est publié.

Trivy arrête une image avec une faille grave qui a déjà un correctif

ci-backend.yaml

```yaml
- name: Trivy image scan
  uses: aquasecurity/trivy-action  # v0.36.0
  with:
    scanners: vuln
    severity: HIGH,CRITICAL
    ignore-unfixed: true
    exit-code: '1'
    trivyignores: .trivyignore.yaml
```



exit-code 1 : le pipeline s'arrête

Cette image ne part pas vers ACR.

Extrait de .github/workflows/ci-backend.yaml (branche staging), lignes simplifiées

Parcours d'un changement · Règle d'équipe

Sans protection de branche, un pipeline rouge n'empêche pas la fusion

Pipeline rouge = un échec

Un contrôle a trouvé un problème. Le job s'arrête, rien n'est publié.

Fusion interdite = protection de branche

Absente ici : offre gratuite de GitHub, dépôt privé.

Pull request : staging → main

✕ Pipeline rouge

bouton actif **Fusionner quand même**

Règle d'équipe : on ne fusionne pas tant que c'est rouge.

Parcours d'un changement · Contrôler ou publier

Une pull request contrôle ; après la fusion, le push sur staging publie

Pull request

- Contrôles des fichiers modifiés
- Si un composant change : image construite et scannée

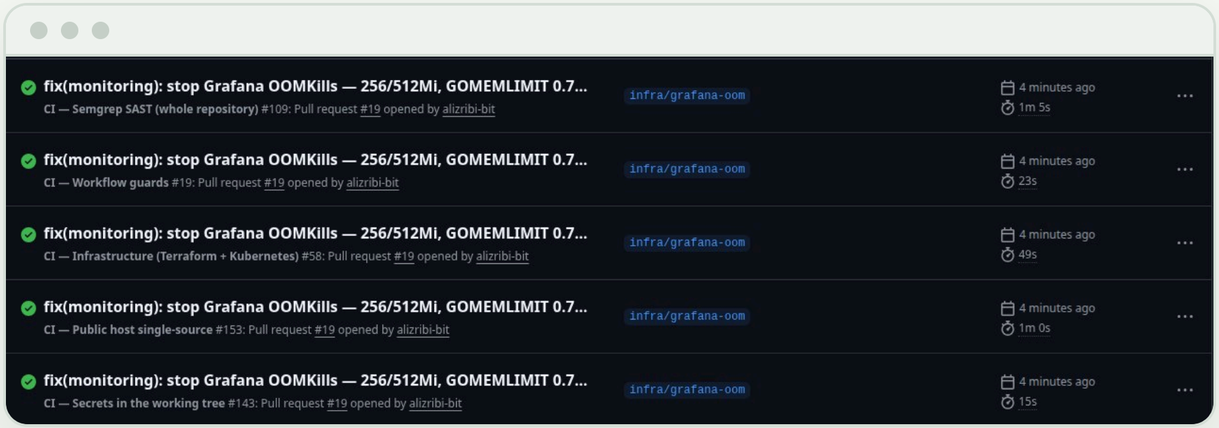
rien n'est publié

fusion →

Push sur staging, après fusion

Si un composant a changé :

- image publiée dans ACR
- manifeste mis à jour



06/10 · PR #19 (configuration du cluster) : contrôles verts, aucune image

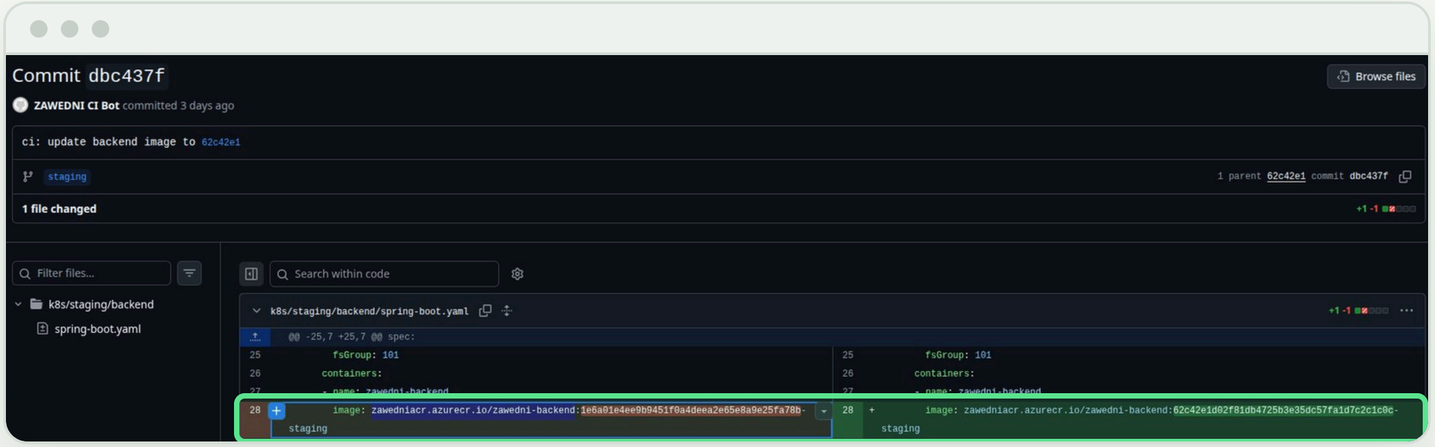
Parcours d'un changement · Publication

Le pipeline publie l'image, puis change une seule ligne dans Git

```
Result
--------------------------------------------------
staging
62c42e1d02f81db4725b3e35dc57fa1d7c2c1c0c-staging
1e6a01e4ee9b9451f0a4deea2e65e8a9e25fa78b-staging
ba2b5ef81377e85b3c814ec92efa67652a358618-staging
```

Capture du 03/10 · registre ACR

ACR = registre d'images ; le nom contient l'identifiant du commit.



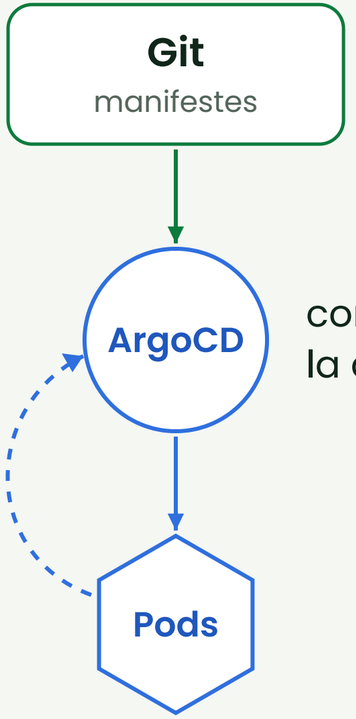
Commit du ZAWEDNI CI Bot · 03/10

Manifeste = fichier Kubernetes qui décrit l'image à lancer.

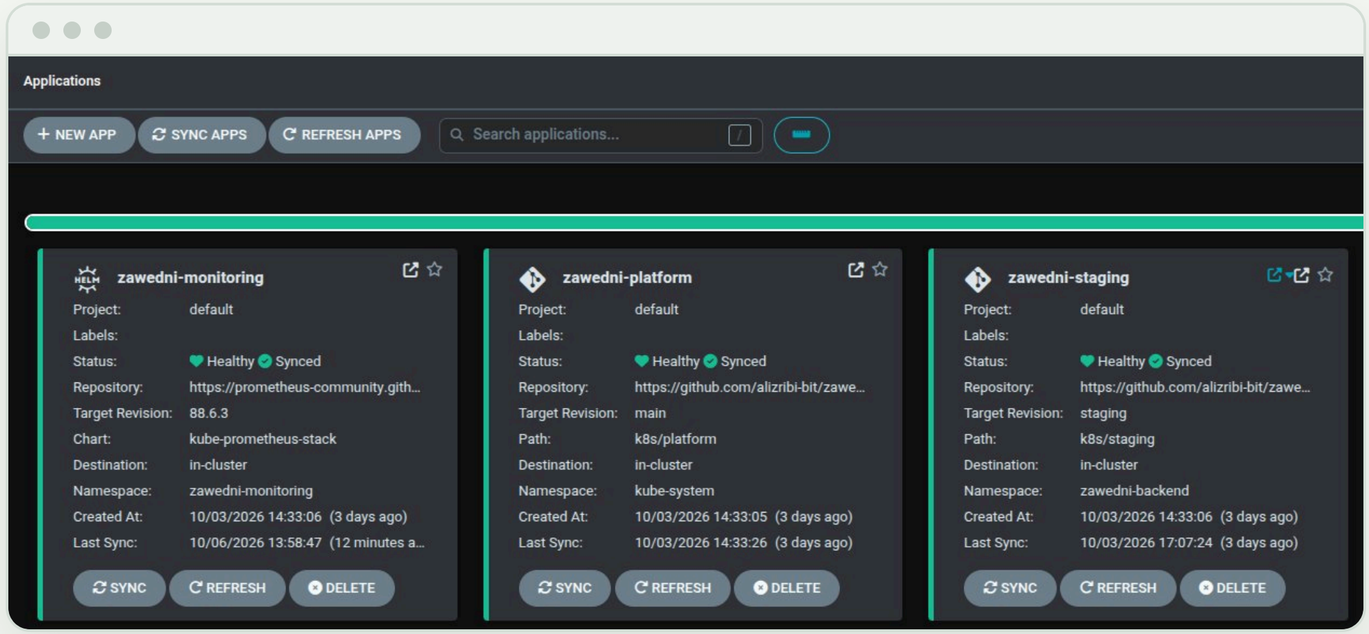
Production : même mécanisme prévu, manifeste encore en :latest

Parcours d'un changement · ArgoCD

ArgoCD applique au cluster la version décrite dans Git



compare et applique la différence

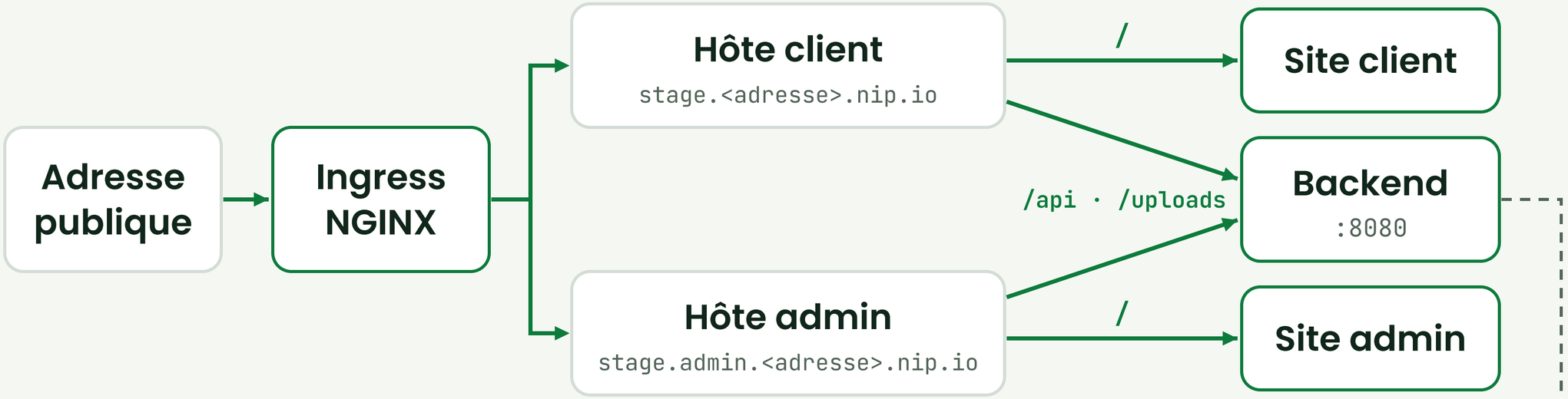


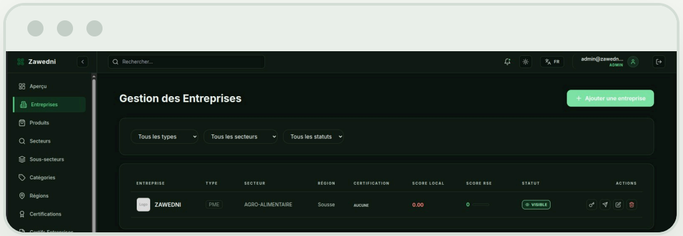
Capture du 06/10 · zawedni-platform, zawedni-monitoring, zawedni-staging : Synced, Healthy

Mises à jour de l'application : le pipeline change le manifeste, ArgoCD applique.

Accès, données et secrets · Routage

L'Ingress envoie chaque adresse vers le bon service

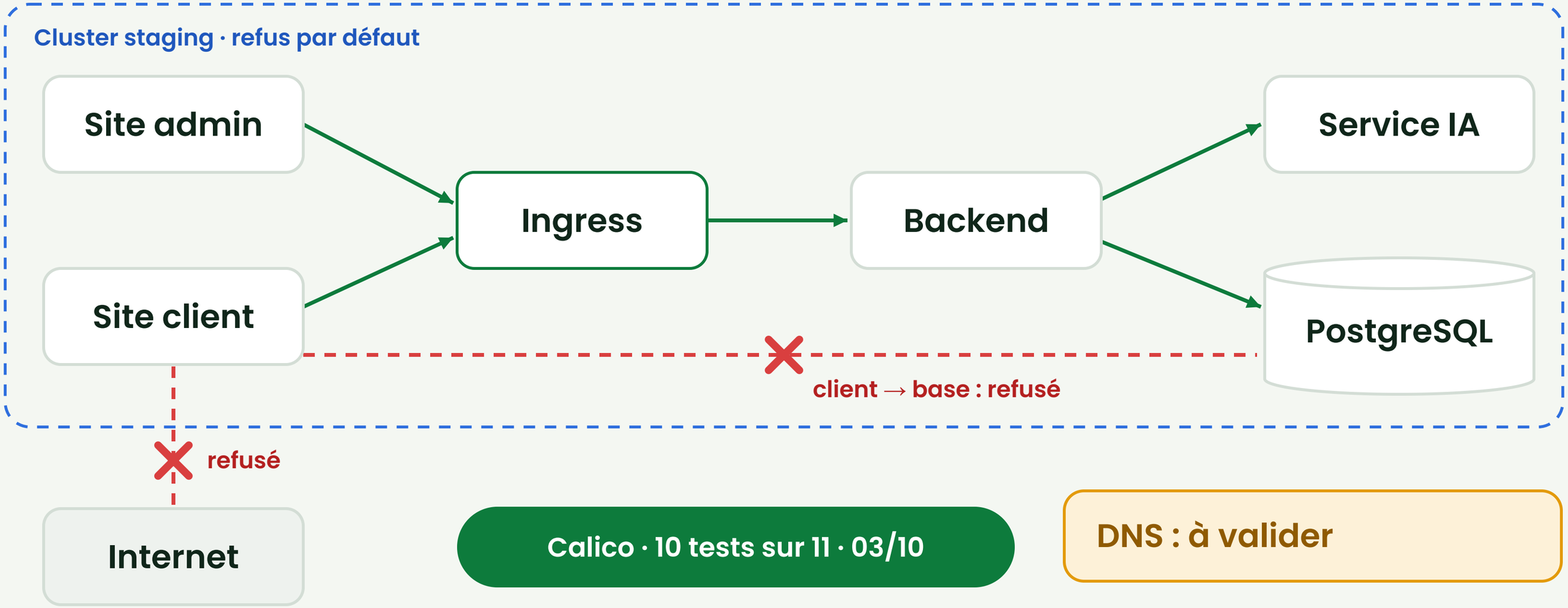




Capture du 06/10 · admin staging

Accès, données et secrets · Isolation

Des NetworkPolicies limitent les échanges entre les pods



Accès, données et secrets · Stockage

Les données vivent sur des disques persistants, et la sauvegarde reste à ajouter

PostgreSQL 16
1 pod

→

Volume 5 Go
`ReadWriteOnce`

→

Disque Azure

Si le pod redémarre, les données restent.

Backend
fichiers envoyés

→

Volume uploads
5 Go · séparé

Un disque persistant n'est pas une sauvegarde
Sauvegarde automatique à ajouter

```
curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://stage.<adresse>  .nip.io/uploads/inexistant.png
HTTP 404
```

Capture du 06/10 · fichier absent → 404

Les secrets ne sont jamais écrits en clair dans Git

Un secret : un mot de passe, une clé ou un token.

Pour le pipeline

Secrets GitHub
`ACR_*, SONAR_TOKEN…`

→

Pipeline
valeur masquée : ***

Pour l'application

Dépôt Git
Sealed Secret
chiffré

Cluster

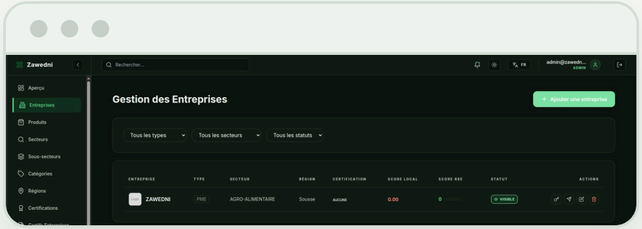
déchiffré seulement ici

→

variable
`ADMIN_DEFAULT_PASSWORD`

→

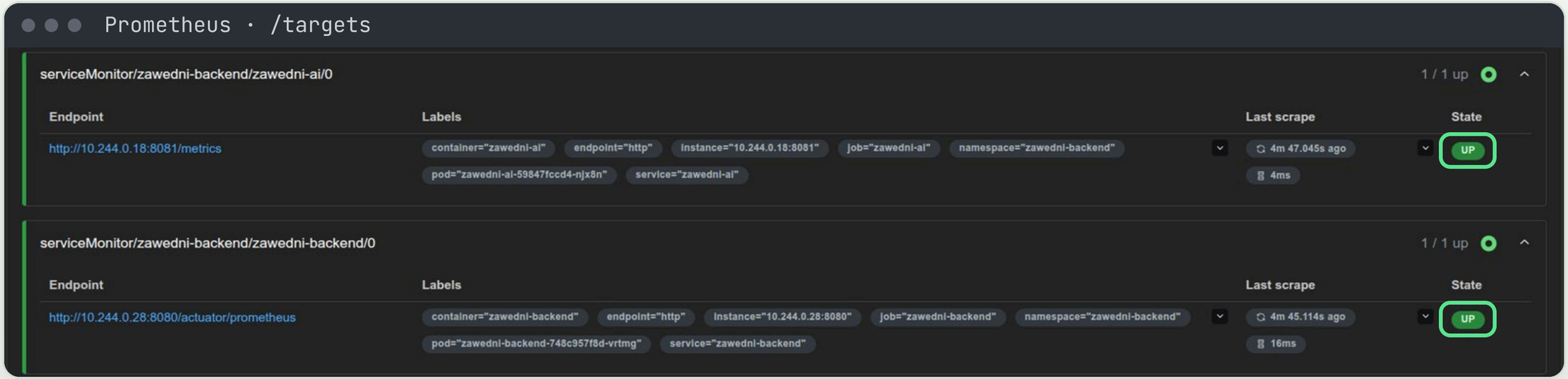
connexion admin



Capture du 06/10 · admin staging connecté

Supervision et résultats · Prometheus

Prometheus collecte les métriques du backend et du service IA

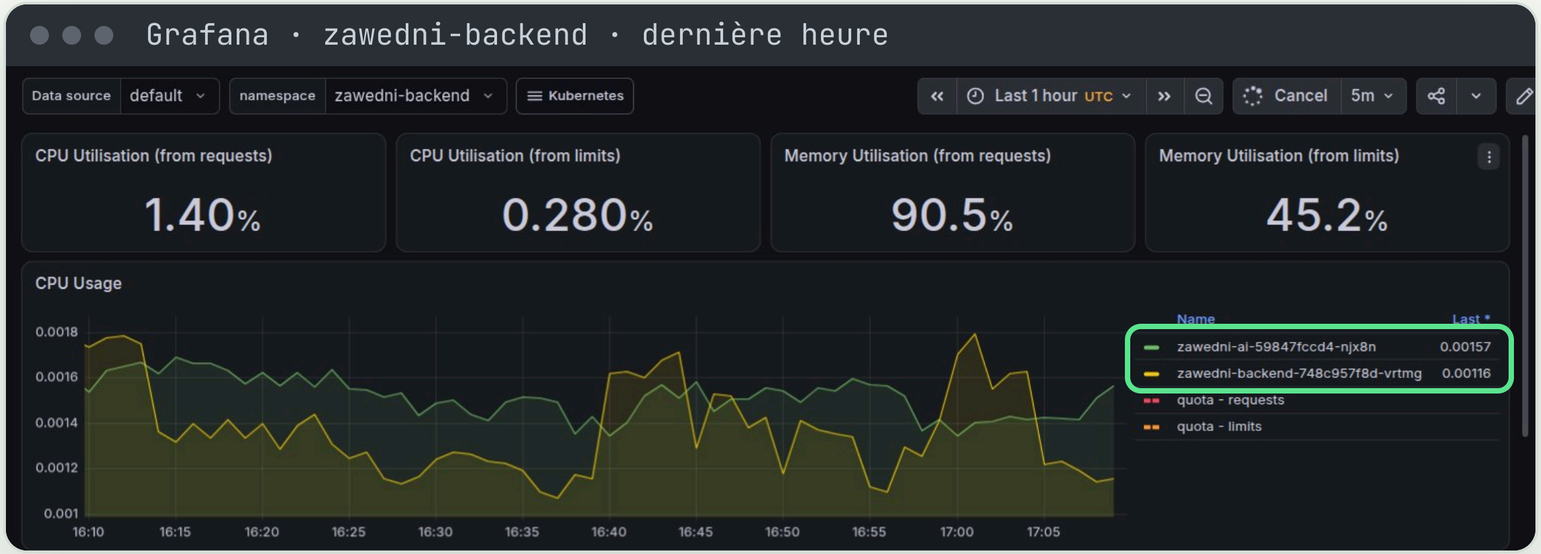


Capture du 06/10 · cibles zawedni-backend et zawedni-ai : UP

Absent du rapport : collecté depuis octobre

Supervision et résultats · Grafana

Grafana montre les ressources réelles des pods du backend et de l'IA



Capture du 06/10 · CPU et mémoire des pods backend et IA, dernière heure

Alertes

Règles écrites, déclenchement non validé

Journaux (Loki)

Non installé

Supervision et résultats · Traçabilité

Le même identifiant relie le commit testé à l'image qui tourne en staging

1 run · 62c42e1 → 2 ACR · 62c42e1 → 3 manifeste · 62c42e1 → 4 pod · 62c42e1

Cluster AKS · staging · zawedni-backend

```
IMAGE                                                                                    READY
zawedniacr.azurecr.io/zawedni-backend:62c42e1d02f81db4725b3e35dc57fa1d7c2c1c0c-staging   true
```

62c42e1 Pod staging · Ready · 06/10

Supervision et résultats · Sprint 6

Le sprint six vérifie si une analyse bloque vraiment, en trois gestes

Bloquer ou signaler ?

Références : **OWASP DevSecOps Guideline** · **NIST SSDF**

1 Lire

la configuration du contrôle

2 Casser exprès

ajouter une erreur exprès

3 Réparer

retirer l'erreur



vert : le pipeline doit repasser

Supervision et résultats · Un essai réel

Dependency-Check a arrêté une bibliothèque log4j vulnérable

1 **Gitleaks** secrets

2 **Tests** JUnit

3 **Dependency-Check** bibliothèques

4 **SonarCloud** qualité

5 **Docker build** construction

6 **Trivy** image

7 **ACR push** publication

Bibliothèque retirée : le pipeline redevient vert

Le rouge venait de log4j.

Essais du sprint 6 : log4j · faux secret · ancienne adresse

Supervision et résultats · Lecture finale

Six types d'analyses : trois bloquent le backend sur staging, une quatrième s'ajoute sur les pull requests et main

Gitleaks
bloquante · testée

Dependency-Check
bloquante · testée

Trivy
bloquante · non testée

SonarCloud
bloque sur PR et main

Semgrep
rapport

Checkov
rapport

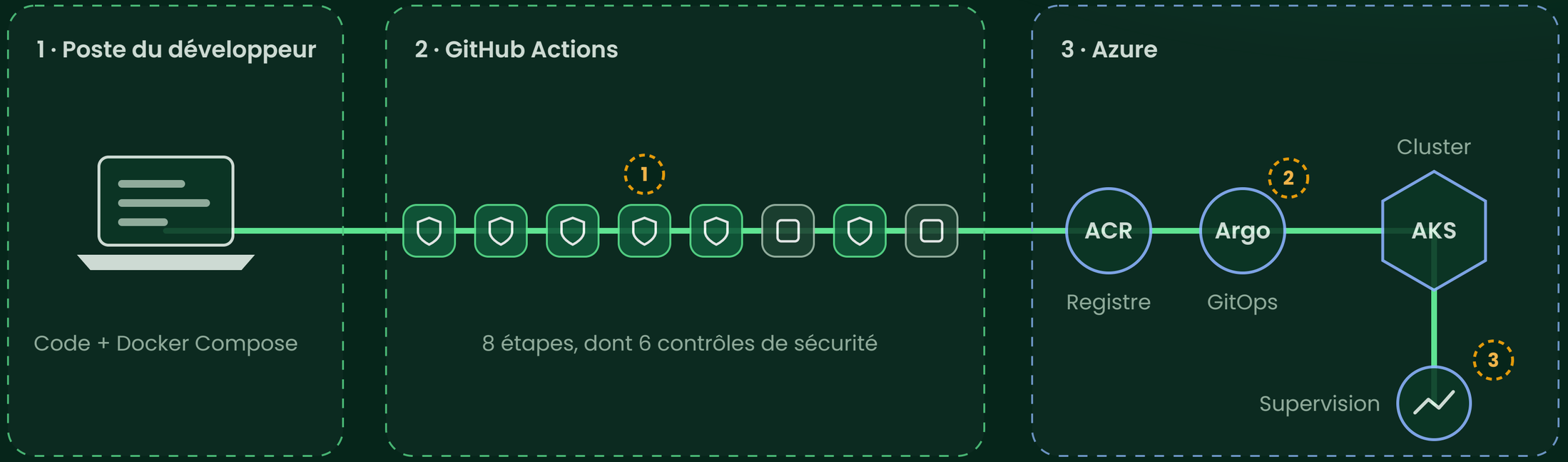
Backend, chemin de staging : **3** bloquantes

Pull requests et main : + quality gate SonarCloud = **4**

Analyses bloquantes : **3** sur staging · **4** sur PR et main

Trois priorités complètent la chaîne

1 Sécurité HTTPS, protection des branches, analyses bloquantes



2 Exploitation production et sauvegardes

3 Supervision alertes et journaux

Entre les sessions de test, on arrête le cluster pour économiser le crédit.

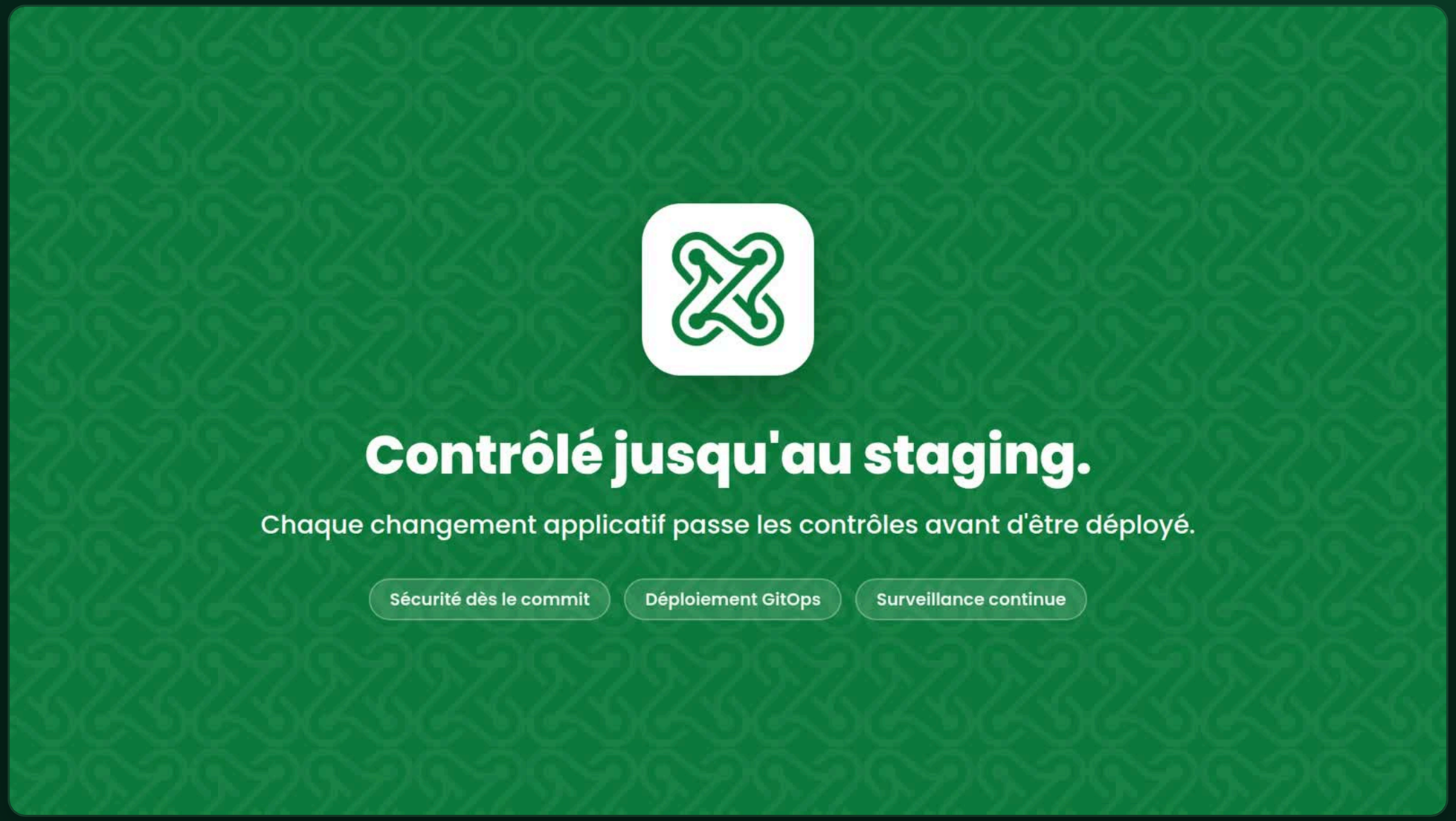
Conclusion · Bilan

En bilan, la plateforme est passée d'un ordinateur à une chaîne cloud contrôlée

	Avant	Après	Statut
Infrastructure	Manuelle	Décrite en code avec Terraform	**Réalisé**
Automatisation	Aucune	10 pipelines	**Réalisé**
Sécurité	0 analyse	6 analyses, 3 bloquantes sur staging	**Réalisé**
Déploiement	Manuel	GitOps, vérifié en staging	**Staging : réalisé** Production : configurée
Supervision	Aucune	Prometheus et Grafana, données réelles	**Réalisé** Alertes : configurées

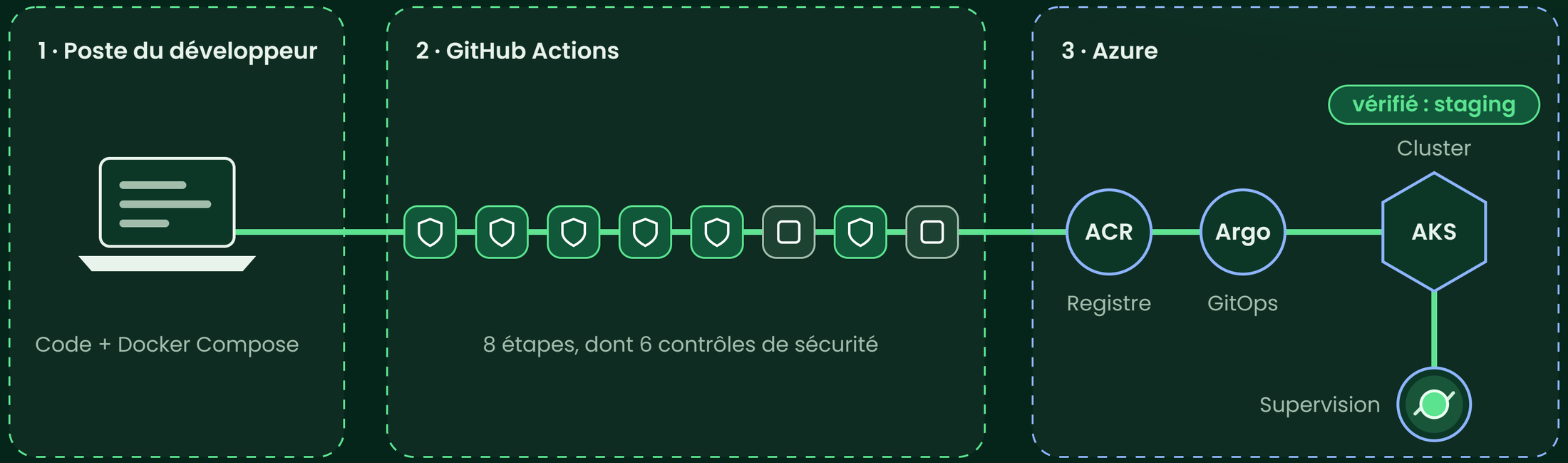
Conclusion · Récapitulatif

Le voyage d'un commit, en une minute trente



Conclusion

Le commit va maintenant jusqu'à staging, contrôlé, publié et déployé sans geste manuel

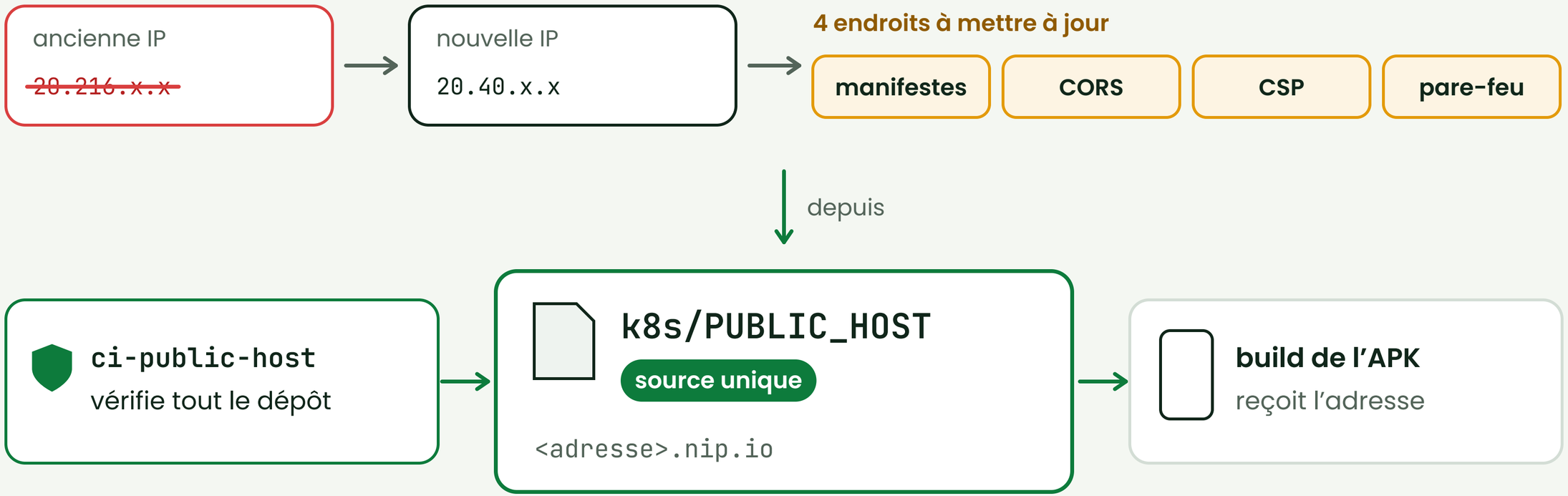


Analyses bloquantes : **3** sur staging · **4** sur pull requests et main

Réserve · Questions

Après un changement d'adresse IP, l'adresse publique est devenue une source unique

Sprint 2 · panne du load balancer Azure, recréé



Réserve · Questions

PostgreSQL garde ses données sur un disque persistant, mais aucune sauvegarde n'est automatisée

Pod · Deployment, 1 réplique

`postgres:16-alpine`

les données ne vivent pas dans le pod

→

Volume persistant (PVC)

5 Go · `ReadWriteOnce`

un seul nœud à la fois

→

Stockage Azure

Disque Azure

les données restent si le pod redémarre

Mot de passe

Dans un **SealedSecret** : chiffré dans Git, ouvert seulement par le cluster

Sauvegarde automatique

Non réalisée

disque perdu = données perdues

Le rapport parle de StatefulSet ; dans les fichiers, c'est un Deployment avec ce volume.

Réserve · Questions

GitOps couvre les mises à jour de l'application ; l'amorçage reste manuel et documenté

Mises à jour de l'application

pipeline → manifeste → ArgoCD

Retour arrière = `git revert`

En staging, chaque image a son identifiant dans Git : possible

Essai réel : non testé

Amorçage

`k8s/bootstrap`, appliqué à la main, dans l'ordre

10-platform → 20-monitoring → 30-staging → 40-prod

> Exemple · 06/10
>
> Correctif Grafana, PR #19 et #20 : fusionné, puis appliqué. **10 min sans redémarrage.**

Leçon des sprints

`kubectl rollout undo` + selfHeal = pods recréés en boucle

Réserve · Questions

Chaque outil a été choisi selon des critères simples, avec le budget comme contrainte

Outil retenu	Critère
GitHub Actions	intégré au dépôt, aucun serveur à entretenir
AKS	plan de contrôle géré par Azure
Terraform	plan relu avant application
ArgoCD	interface, retour par Git
Trivy	scan de l'image dans le pipeline
Prometheus + Grafana	métriques et tableaux de bord

Crédit Azure

100 $

Estimations du rapport

- cluster : **35–40 $ / mois**
- ACR Basic : **≈ 5 $ / mois**

Aucune mesure de consommation réelle
le cluster est arrêté entre les tests

Autres solutions possibles :
Jenkins, un serveur CI à entretenir ;
GitLab CI et Azure Pipelines, compatibles avec GitHub, un service de plus à configurer.

Réserve · Questions

Ce qui tourne dépend de l'événement : push, pull request, branche et fichiers modifiés

Événement	Ce qui tourne	Publication
push feature, develop	workflows démarrés, jobs sautés ; seuls les guards, si workflows ou `k8s/` changent	non
PR → develop	Semgrep ; mobile si `zawedni/` change ; guards	non
PR → staging, main	composants touchés + transversaux ; image construite et scannée	non : rien publié
push staging	mêmes contrôles, sans attendre la gate SonarCloud	**image `:<sha>-staging` + manifeste staging**
push main	mêmes contrôles + gate SonarCloud attendue	prévu : `:<sha>-latest` + manifeste prod, non démontré

Contrôles par workflow

backend	Gitleaks · JUnit · Dependency-Check · SonarCloud · Trivy
client, admin, ia	Gitleaks · tests · Trivy
mobile	Gitleaks · flutter test · APK
transversaux	secrets · public-host · guards · Semgrep, Checkov : rapport

Trois essais réels, avec une vraie erreur

- log4j 2.14.1 → Dependency-Check
- faux secret → Gitleaks
- ancienne adresse → public-host

Réserve · Questions

Le mobile est testé et construit en APK par la CI ; l'upload Drive n'est pas prouvé

ci-mobile, à chaque run

Gitleaks → **flutter test** → **build APK** adresse injectée depuis `PUBLIC_HOST` → **artefact GitHub**

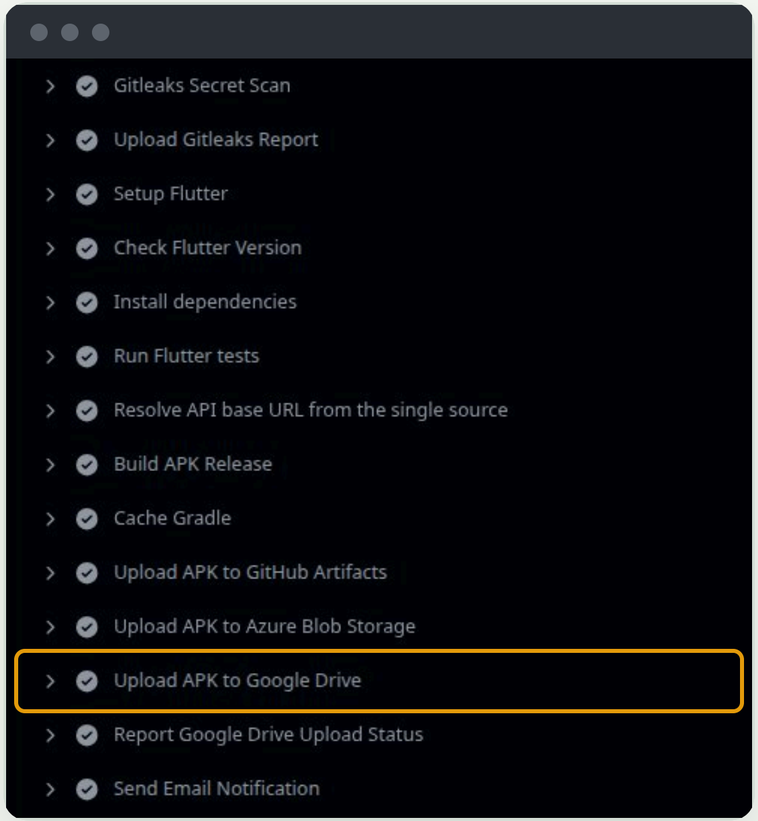
Sur un push, en plus

Azure Blob
`apk-releases`
nom = date + SHA

→ **e-mail**

Google Drive : non bloquant · étape verte en 0 s · upload non prouvé

Démonstration sur téléphone : après la fin des changements d'interface.



Run du 03/10 · build-apk réussi en 15 min 25 s · encadré : Drive

Réserve · Questions

Sur 22 exigences, l'état démontré est connu pour chacune

Fonctionnelles

BF01 Conteneurisation	**BF02** Environnement local Compose	**BF03** Provisionnement Terraform	**BF04** Déclenchement par composant	**BF05** Contrôles bloquants
BF06 GitOps en staging	**BF07** Isolation des environnements	**BF08** OWASP ZAP	**BF09** Supervision des métriques	**BF10** Journaux Loki
BF11 Alertes	**BF12** Falco	**BF13** Secrets	**BF14** APK dans Azure Blob	**BF15** Communications et images

Non fonctionnelles

BNF01 Performance	**BNF02** Disponibilité 99,9 %	**BNF03** Scalabilité (HPA)	**BNF04** Maintenabilité	**BNF05** Portabilité	**BNF06** Traçabilité	**BNF07** Coût

démontré · configuré ou partiel · non réalisé

Total : 8 · 9 · 5

Réserve · Questions

Le rapport, le code au même commit et le travail d'après sont trois états différents

1 · Rapport

699b99f, 7 septembre

- SonarCloud non bloquant
- secrets Kubernetes
- aucune métrique ZAWEDNI
- NetworkPolicies non appliquées

2 · Déjà dans le code

au même commit, non décrit

- quality gate SonarCloud attendue sur PR et main
- Sealed Secrets

3 · Ensuite

après le rapport

- premier commit du bot sur staging : 3 oct.
- cluster 1 nœud : staging + supervision
- métriques backend et IA collectées
- Calico : 10 tests réseau sur 11
- uploads : 404 vérifié, 06/10
- Grafana stabilisé, PR #19 et #20, 06/10
- Loki non réinstallé

Reste ouvert : production GitOps non démontrée · alertes non validées · sauvegardes · ZAP · Falco / OPA